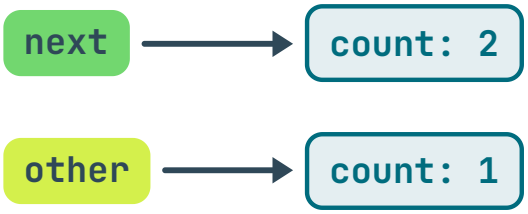
Closures

A function plus the scope it was born in

```
function makeCounter() {
  let count = 0;
  return () => ++count;
}
const next = makeCounter();
const other = makeCounter();
next();   // 1
next();   // 2
other();  // 1
```

What stays in memory



Each call of **makeCounter()** creates its own private **count**.

1 Factory

```
const add = a =>
  b => a + b;
const add2 = add(2);
add2(5);    // 7
add2(10);   // 12
```

2 Private state

```
const w = (() => {
  let cash = 10;
  return () => cash;
})();
w();      // 10
w.cash; // undefined
```

3 Loop pitfall

```
var i      3 3 3
```

one shared **i**

```
let i      0 1 2
```

a fresh **i** per turn

in **setTimeout** callbacks

TRAP What does f() return?

```
let x = 1, f = () => x;
x = 2;
console.log(f());
```

2

A closure captures the variable, not its value.

KEY TAKEAWAY

- **closure** — Function + its scope
- **live** — Sees later changes
- **memory** — Keeps data alive

? Follow-up questions to expect

- **What is a closure in one sentence?** → A function plus the variables of its birth scope.
- **Does it copy values or variables?** → Variables: it sees later changes.
- **Do two closures share variables?** → Same outer call: yes. New call: new copy.
- **Can closures keep memory alive?** → Yes, while the function is reachable.